

Devoir Surveillé
Documents interdits
2 heures

Exercice 1 (Questions de Cours (3 pts))

- Expliciter les caractéristiques de la génération d'ordinateurs 1965-1980 (1pt).
- Définir la notion de pagination (1pt).
- Que fait la fonction *lseek(int, off_t, int)* (1pt) ?

Exercice 2 (Processus et répertoires et tubes (11 pts))

1) Soit le programme source suivant :

```
#include <stdio.h>
extern int e = 423;
int main(int argc, char* argv[]){
    int p = 3;
    int t = 3;
    for(int j = 0 ; j < 2 ; j++){
        t = fork();
        if (t){
            p = 421;
            e = 668 ;
            printf("%d \t %d \t %d \n", e, p, t);
        }
    }
    printf("%d \t %d \t %d \n", e, p, t);
    execl("/bin/ps", "ps", NULL);
    printf("%d \t %d \t %d \n", e, p, t);
    printf("--Fin-- \n");

    exit(0);
}
```

Expliquer l'affichage réalisé par ce programme.

2) Ecrire un programme qui va exécuter une commande passée en paramètre un nombre de fois passé en paramètre (*./prog ps 3* exécutera 3 fois la commande *ps*). L'affichage devra être cohérent (*i.e.* pour la commande précédente, l'affichage sera identique à 3 *ps* séquentiels).

3) Ecrire un programme qui envoie des entiers positifs (saisis par l'utilisateur) à son processus fils via un tube. Les deux processus s'arrêteront proprement quand un entier négatif aura été saisi par l'utilisateur.

4) Ecrire un programme qui liste les fichiers du répertoire courant.

Exercice 3 (Ordonnancement (6 pts))

Soit 5 processus P1, P2, P3, P4 et P5 de durée P1 : 9s, P2 : 8s, P3 : 6s, P4 : 10s et P5 : 13s soumis aux temps P1 : t = 0s, P2 : t = 1s, P3 : t = 2s, P4 : t = 4s et P5 : t = 6s. Ces 5 processus ont respectivement les priorités 3,1,3,5 et 5 (5 représentant la priorité maximale). Pour chacun des algorithmes d'ordonnancement

suivants, donnez le temps moyen d'attente (temps d'attente : temps nécessaire pour que le processus soit élu pour la première fois par l'ordonnanceur) et la durée moyenne d'exécution (temps d'exécution : temps écoulé entre la soumission et la fin de l'exécution).

FIFO

Job le plus long en premier

Job le plus court en premier

Au plus court reste

Round Robin (Quantum=2s)

Ordonnancement par priorité (Quantum=1s)

Annexes

```

— int close(int fd);
— int closedir(DIR *dir);
— int chdir(const char *file);
— int execlp(const char *file, const char *arg, ... /*, (char *)0 */);
— int fork();
— off_t lseek(int fd, off_t offset, int whence);
— int open(const char *pathname, int flags, mode_t mode);
— DIR *opendir(const char *name);
— int pipe(int filedes[2]);
— int printf(const char * restrict format, ...);
— int rand(void);
— ssize_t read(int fd, void *buf, size_t count);
— struct dirent *readdir(DIR *dir);
— void seekdir(DIR * D, off_t pos);
— void (*signal(int sig, void (*func)(int)))(int);
— void srand(unsigned seed);
— int stat(const char *path, struct stat *buf);
— int raise(int signo);
— off_t telldir(DIR * D);
— ssize_t write(int fd, const void *buf, size_t count);
— int wait(int * status);
— struct stat {
    dev_t st_dev; /* ID du périphérique contenant le fichier */
    ino_t st_ino; /* Numéro inoeud */
    mode_t st_mode; /* Protection */
    nlink_t st_nlink; /* Nb liens matériels */
    uid_t st_uid; /* UID propriétaire */
    gid_t st_gid; /* GID propriétaire */
    dev_t st_rdev; /* ID périphérique (si fichier spécial) */
    off_t st_size; /* Taille totale en octets */
    blksize_t st_blksize; /* Taille de bloc pour E/S */
    blkcnt_t st_blocks; /* Nombre de blocs alloués */
    time_t st_atime; /* Heure dernier accès */
    time_t st_mtime; /* Heure dernière modification */
    time_t st_ctime; /* Heure dernier changement */
};
— struct dirent {
    ino_t d_ino; /* numéro d'inoeud */
    off_t d_off; /* décalage jusqu'à la dirent suivante */
    unsigned short d_reclen; /* longueur de cet enregistrement */
    unsigned char d_type; /* type du fichier */
    char d_name[256]; /* nom du fichier */
};

```